

Propunere Presentare: Arhitectură Hibridă de Control Distribuit via SSH

Furtună Denis-Andrei

1. Motivația Proiectului

Acest proiect s-a născut din dorința de a duce proiectul de la laboratorul Sisteme de Operare la alt nivel. Obiectivul meu principal a fost să transform o implementare locală, destinată exclusiv evaluării în terminal, într-un sistem distribuit „Production-Ready”. Am scalat aplicația construind o interfață de control de la distanță complet asincronă. Astfel, am făcut o logică de bază de gestiune a proceselor controlată pe rețea pentru a putea fi utilizată de un End User, transformând o temă de laborator într-un proiect viabil pentru portofoliul meu profesional.

2. Încadrarea în Categorie și Argumentare

Categoria bifată: un proiect software care folosește tehnologii distribuite, tehnologii de comunicare la distanță, tehnici de sincronizare a thread-urilor și/sau proceselor etc.

Proiectul reprezintă o arhitectură Client-Server eterogenă (sistemele client și server sunt pe sisteme de operare diferite). Am utilizat proiectul standard de la laborator exclusiv ca Backend/Server. Contribuția mea în plus constă în dezvoltarea unui Control Panel distribuit pe Windows, care orchestrează acest backend de pe Linux.

Comunicarea la distanță este securizată printr-un tunel SSH (TCP/IP). Elementul central este tehnica de sincronizare: pe partea de client am implementat Multi-Threading concurent pentru ca operațiunile de rețea să nu blocheze interfața grafică. Pe server, am orchestrat procesul Daemon prin semnale sistem asincrone, rezolvând paralelizarea și latențele comunicării inter-platformă.

3. Documentație Tehnică

A. Componenta Client (Control Panel - Windows/Python):

- **Paramiko:** Pilonul central al comunicării, utilizat pentru stabilirea tunelului SSH peste TCP/IP și executarea comenzilor (RPC) în mod securizat pe server.
- **Threading:** Implementat pentru a asigura o interfață complet asincronă, permițând ascultarea continuă a log-urilor (tail -f) pe fire de execuție separate, fără a bloca interacțiunea utilizatorului.
- **Time:** sleep pentru terminarea scrierilor pe disc.
- **CustomTkinter:** Framework-ul utilizat pentru dezvoltarea interfeței grafice (GUI), oferind o arhitectură modernă și modulară pentru monitorizarea terminalelor.
- **Dotenv / OS:** Utilizate pentru managementul securizat al variabilelor de mediu și ascunderea credențialelor SSH (principiu de bază în securitatea aplicațiilor).

B. Componenta Server (Motor de Execuție - Linux/C):

- **POSIX API (unistd.h, sys/stat.h, fcntl.h):** Utilizat intensiv pentru manipularea low-level a sistemului de operare: managementul proceselor (fork, familiile exec), controlul atomic al fișierelor (O_APPEND, manipulare Inode-uri, Symlinks) și comunicare Inter-Process (Pipes).
- **Signal Handling (signal.h):** Gestiunea avansată a întreruperilor asincrone prin structura sigaction, folosită pentru oprirea controlată a daemon-ului de monitorizare.

4. Schița Prezentării

1. Introducere și Viziune Arhitecturală

- De ce o interfață grafică și nu un terminal negru?
- Transformarea unei teme de laborator într-un produs pentru utilizatorul final (End-User).

2. Arhitectura de Ansamblu a Sistemului

- Client Windows (Python) → Server Linux (C).
- Separarea totală a volanului (interfața) de motor (execuția din spate).

3. Tunelul SSH și Comunicarea la Distanță

- Cum trecem granița dintre două sisteme de operare complet diferite?
- Utilizarea Paramiko: comenzi trimise invizibil și securizat pe TCP/IP.

4. Sincronizarea Asincronă (Multi-threading)

- Cum prevenim înghețarea butoanelor la latențele de rețea?
- Utilizarea firelor de execuție (Background Threads).

5. Provocări Critice și Soluții Tehnice

- Client SSH vs. Daemon

6. Demonstrație Aplicație (Live Demo)

- Rularea sistemului cap-coadă.